

DEMYSTIFYING WEB3: BUILDING A TODO APP WITH SOLIDITY AND JAVASCRIPT

PART 1: BUILDING AND DEPLOYING A SOLIDITY SMART CONTRACT
TO THE ETHEREUM BLOCKCHAIN.

AUTHOR: **ABDULHAKEEM MUHAMMED IBIYEMI**
(BLOCKCHAIN/CRYPTOCURRENCY/SMART-CONTRACT DEVELOPER,
SOFTWARE ENGINEER, DATA ENGINEER, AI/ML ENGINEER)

PROJECT REPO LINK: <https://github.com/ennas-de/ToDoApp-Web3>

PROJECT LIVE LINK: <https://todoapp-web3.netlify.app/>

ARTICLE LIVE LINK:

<https://docs.google.com/document/d/1J3dpmh42kGvumS1ucBowW7K3fVevkHQW>

TABLE OF CONTENTS:

1. INTRODUCTION TO WEB3

- 1.1 Introduction to the Internet
- 1.2 Differences between Web3 and Web2
- 1.3 What is a Blockchain (overview)
- 1.4 What is Solidity
- 1.5 Why Solidity?
- 1.6 Introduction to DApps
- 1.7 Ethereum Virtual Machine (EVM) (overview)

2. SETTING UP DEVELOPMENT ENVIRONMENT

- 2.1 Development Environments (Testnet vs Mainnet)
- 2.2 Smart Contracts security vulnerabilities (overview)
- 2.2 Development tools and libraries

3. DEVELOPING A TODO APP SMART CONTRACT USING SOLIDITY PROGRAMMING LANGUAGE

4. DEPLOY THE SOLIDITY SMART CONTRACT

- 4.1 Deploy our TODO App Smart Contract to the Sepolia Testnet (using Hardhat on the CLI)
- 4.2 Verifying our Smart Contract on the Etherscan using the CLI
- 4.3 Verifying our Smart Contract on the Etherscan block explorer using a Browser

5. CONCLUSION

- 5.1 Interacting with our TODO Smart Contract on the Backend
- 5.2 Summary and Conclusion
- 5.3 What's Next (Part 2: Connecting to our TODO Smart Contract from the Client Side using React, JavaScript, and Web3.js)

1. INTRODUCTION TO WEB3

1.1 Introduction to the Internet

The Internet has become a common word in our day to day life and activities. It is defined as 'a vast network that connects computers all over the world' (Britannica.com). In simpler terms, it simply refers to computers, phones, tvs, and every form of gadget that are connected to one another throughout the world.

With our understanding of the Internet, we now understand that different devices are in one way or another connecting to one another through a 'means'. One of these medium of connection is the World Wide Web. (also known as The Web, WWW, or W3) (techtarget.com).

The World Wide Web or The Web refers to all the public websites or pages that users can access on their local computers and other devices through the internet (Techtarget.com). These websites or pages are running on a remote computer (servers) that have access to and are connected to the internet. To access this websites, a user need to know the location (ip address) of the website of interest. Each individual website must have their individual and unique IP address (for example 157.240.20.35, is a very popular website that has millions of users visiting it per month).

But unlike computers that can store up and look for high volumes of data and also perform high levels of computations, human beings are more of the reasoning type, hence, we prefer a more easy to understand and remember format. This leads to the development of the DNS (Domain Name System). The domain naming is a format for mapping an IP Address(es) to a human readable format (facebook.com is easier to remember and understand than 157.240.20.35, right?).

Web 1.0 is the first generation (or version) of the WWW, and it is a very boring, static format. It is not interactive as it is 'read-only'. Web2 came in as a champion and brought about a lot of modern day features and functionalities (like interactions, responsiveness, etc). Web2 is the current format of the internet we are all aware of (running mostly on HTTP2.0). Platforms like Google, Facebook, and others, are big players in the Web2 world.

Social interactions are made possible with Web2, and one of the key features of Web2 is the 'Centralization' of information or data. This in simpler terms is just the fact that for every Web2 platform, the platform owner holds ALL the records and information of happenings on the platform. They control events and users on their platforms. This has its Pros (regulating contents) and Cons (Security, Trust, Data breaches, Data Selling, etc). All these challenges and more led to the development and advancement of Web3 - A Decentralized Technology for the WWW internet connected devices (called Nodes).

Web3 is a decentralized WWW that leverages the Blockchain technology to run. Unlike

the Web2 where individual platform owners holds all the cards, Web3 (leveraging the Blockchain) allows for the possibility of every user having access to the data or information of that platform. Web3 allows for multiple computers to hold the records of data for their platform, and these data is accessible and available to ALL interested parties or nodes (this depends on the type of Blockchain used, either open, private, or hybrid). The participating individuals also holds the power to determine the functioning of their platform (governance), provides security to their platform, and ensure all data are valid and shared among all participating computers (or nodes).

1.2 Differences between Web3 and Web2

Web3 and Web2 are two different (or distinct) phases of the WWW. Each holding different ideologies and key strengths and weaknesses.

Lets take a look at the some of the features of Web3 in comparison to Web2.

A. Ownership and Control:

- a) Web3 at its core is all about DECENTRALIZATION. Web3 introduces the removal of a single entity dictatorship for data and affairs, and introduces a distributed form of consensus and governance.
- b) Web2 has its core in a centralized and controlled system where data and governance are decided by the underlying platforms.

B. Interactivity and Transactions:

- a) Web3 leverages the Blockchain (distributed) technology, allowing for a peer-to-peer form of interactions or transactions among peer (without the need for an intermediary)
- b) Web2 provides a middle man. Every event, interaction or transaction must firstly pass through the predefined middle man before getting to the final destination (example, financial transactions in Web2 involves intermediaries like banks, etc).

C. Identity:

- a) Web3 introduces the concept of 'WALLETS', and every interaction is through each individual wallet. A user that is interested in access or leveraging the Web3 ecosystem must do through a certain Blockchain Wallet. These Wallets are cryptographically generated and are unique. They can hold a form of asset that is allowed on the Blockchain (often used in Cryptocurrencies). Not all Blockchains are for cryptocurrencies.
- b) Web2 identity is provided through a unique username and password, or a combination of other means and methods depending on the platform.

D. Data Privacy:

- a) Web3 data are open to the world to see (for open blockchains, or closed for private blockchains, or a mix of both for hybrid blockchains). Every interaction in a blockchain (regardless of type) that lead to a change of state is encrypted and validated before it can be allowed to be added to the blockchain.
- b) Web2 on the other hand has its data stored in a secure backend database. Some of these data are encrypted (example, user passwords, etc), but the transaction or interactions are not encrypted, hence, any malicious data is allowed into the system. This leads to a lot of data breaches.

1.3 What is a Blockchain? (Overview)

We've heard so much about BLOCKCHAIN and how they are the fundamental technology behind Web3. So what is the Blockchain technology?

In layman's terms, 'BLOCK' is like a foundational building unit of a structure, and 'CHAIN' is a link or connection of things. This same concept is applicable in Blockchain technology. So, Blockchain technology is a technology that is characterized by a combination 'or chaining' or BLOCKS.

Blocks are the foundational, and the most important aspect of the Blockchain technology. The Blocks of a Blockchain holds all the data or transaction available on that Blockchain. When a transaction occurs in a Blockchain, the transaction can either be changing the state of the Blockchain (by adding new Blocks to the Blockchain), or the transaction can simply be querying the Blockchain (by requesting data from the Blockchain).

We would be discussing Blockchain technology in detail and build our very own Blockchains in later editions. We would build Custom Blockchains, by building a full blockchain solution from scratch using JavaScript, and also by using a template to build a Blockchain using the Rust programming language.

Data on a Blockchain can be accessed by participating entities (nodes or peers). To put it simply, a Blockchain is a decentralized ledger that stores data across a network of computers (which are also called peers or nodes).

There are 4 Layers to the Blockchain Technology:

* **Layer 0:** This is the Infrastructure layer. It is the foundational layer that consists of hardware, network protocols, and others. The Polkadot Blockchain is an example. More explanation on this when we build our own Blockchain in Rust using the Polkadot SDK (formerly Substrate).

- * **Layer 1:** This is the Base Blockchain Protocol. Layer 1 Blockchains are the typical Blockchains everyone is aware of and used to. Examples of Layer 1 Blockchains include Bitcoin, Ethereum, among others. Layer 1 Blockchains are responsible for the core features of the Blockchain that the users can interact with. Transactions validation and Consensus mechanisms are some of the features Layer 1 Blockchain provide.
- * **Layer 2:** Layer 2 Blockchains are like an 'advanced feature' Blockchain. They are built as a form of ENHANCEMENT for their parent Layer 1 Blockchain. The Layer 2 Blockchains are created to resolve several issues affecting their Layer 1 Blockchains. Some of these features added by a Layer 2 Blockchain include scalability, speed, and reduced transaction fees or costs. Example of Layer 2 Blockchains include .
- * **Layer 3:** Decentralized Applications (dApps) are present in this layer. This layer provides the functionality for creating, owning and interacting with the Blockchain assets (Tokens, NFTs, etc). This layer is where the Ethereum Smart Contract functionalities operate in.

1.4 What is Solidity?

Solidity is a programming language that was created for interaction with the Ethereum Virtual Machine. The language is used to write codes (smart-contracts) that interacts with the Ethereum Virtual Machine (EVM). Solidity allow developers to write decentralized applications (dApps) that are secure, fast, and self-executing on the Ethereum Blockchain.

Solidity language is a statically typed language that is an object-oriented language.

1.5 Why Solidity?

I've written many dApps' smart-contracts, I have also interacted with many smart contracts, but like always, I wanted an understanding of the Solidity language. I was puzzled with the need for Solidity to interact with Ethereum Blockchain Layer 3 features.

I had these questions on my mind that made me do more indepth research on the Ethereum Blockchain, and Solidity:

- A. Why did the Ethereum co-founder create a new programming language to interact with the Ethereum VM?
- B. Why did they not use an existing programming language to interact with the EVM?
- C. Why was the Solidity core logic structured the way it is?

D. Why do the Solidity developers separate the smart-contract from the core of the blockchain?

Here is my simple understanding of Solidity after much research:

“Every programming language has its focus. Some languages are more targeted to web development (JavaScript, TypeScript, etc), System Development (C, Cpp, C#, etc), VM (like Java), and multiple (like Python, Dart, etc).

Each of these languages depending on where they are primarily targeted to run, determines their core components and built-in features. So in Solidity programming language, everything about its internalization is made for the Ethereum Virtual Machine and dApps.

Security, Efficiency, Application use, Speed, Features, Abstraction, keywords, and Syntax, etc, of the Solidity language are all made for the Ethereum EVM.”

1.6 Introduction to dApps

Decentralized Applications (dApps) are applications or software programs that are built to interact with a Blockchain technology.

These applications leverage the blockchain technology to perform their core functionalities. DApps can either run a complete web3 solution (without running any server, as all their backend logic is ran through the blockchain), or include a backend logic with a centralized endpoint.

The dApps have their logic defined on a Smart Contract that has been developed for the application. This smart-contract serves as the backend for the dApp, and all interactions on the platform are ran through the smart-contract attached.

1.7 Ethereum Virtual Machine (EVM)

Ethereum Virtual Machine (EVM) is an engine that executes smart contract codes. It is built to interact with smart contracts and allow code executions in a sandboxed environment.

The EVM is a multiplatform application that allows for smart contracts written in Solidity to run. EVM is used by smart contracts written for the Ethereum, Polygon, and other Blockchains. These Blockchains have some similarities (from my experience so far they are Blockchains built with the Polkadot SDK, formerly Substrate).

2. SETTING UP DEVELOPMENT ENVIRONMENT

2.1 Development Environments (Testnet vs mainnet)

In a Blockchain ecosystem, transactions (ones that change the state of the blockchain) is expensive. During development, a lot of calls would be made to the smart contract to test functionalities and features.

Developing all these on a real blockchain is extremely costly, and would require a lot of changes or calls that would need to be discarded in production environment. This leads to the development of test networks (testnets) for several blockchains (main networks or mainnets) that support smart contracts. On this test network a developer can build, deploy, and test their various smart contracts and see how they will perform on live (mainnet) network.

In summary, the Mainnet is the fully operational and live blockchain, while the Testnet is a testing environment that mimics the mainnet's structure and operation. Real transactions occur in the Mainnet, while the testnet operations and assets are only for testing purposes.

Examples of Ethereum Testnets:

- A. Sepolia
- B. Goerli, etc.

The Polkadot Blockchain also has the Westend network

2.2 Some Smart Contract security vulnerabilities (overview)

Every application or software or technology has its strengths and weaknesses. Smart contract security risks are very serious as Solidity smart contracts can hold funds (in millions), and any slight mistake can lead to a loss of these funds.

Here are some examples of smart contract security vulnerabilities or risks:

- A. Reentrancy Attacks: This type of vulnerability is a very easy one to commit, and it allows for an illegal transfer of (complete) funds from a smart contract.
- B. Frontrunning: When you interact with a smart contract, especially transactions that change the state of the blockchain, you include an estimated gas fee. When this transaction is sent to the Ethereum transaction pool, they become visible to everyone. This allows miners especially to see the smart contract, copy your transaction (if it's a profitable one) and send theirs with a higher gas fee. This (with a higher gas fee) no

have the potential to be picked up faster than your initial transaction. This is especially seen in cryptocurrency tradings.

2.3 Development Tools and Libraries

A. Visual Studio Code: VSCode is my preferred choice of IDE for Blockchain and web3 related developments.

B. Programming Languages:

- a) Solidity: For smart contract development on that run on the EVM, we use Solidity as the programming language of choice.
- b) JavaScript: JavaScript is my deployment scripting language. Deployment codes for the Solidity Smart contract will be written in JavaScript (you can use other languages like TypeScript).

C. Some of the Installed Libraries:

- a) Hardhat: As stated earlier, we need a development environment that can interact with the EVM. Hardhat is one of these development environments, and we use it for compiling, debugging, and deploying the smart contracts and dApps.
- b) Others include: Nomiclabs, NomicFoundation, Openzeppelin, dotenv, ethereum-waffle.

3. DEVELOPING A TODO APP SMART CONTRACT USING SOLIDITY PROGRAMMING LANGUAGE

We will now go through a step-by-step process of building our smart contract from scratch.

STEPS:

1. Make the project directory and move into this director

```
$ mkdir todo-smart-contract
```

```
$ cd todo-smart-contract
```

2. Setup the project directory: Initialize Git and NPM

```
$ git init
```

```
$ npm init -y
```

3. Setup your .gitignore file

a) Create a .gitignore file

b) Open the file and add ``node_modules`` and ``*env*`` to the file.

c) Save and close

4. Install necessary dependencies:

```
$ npm install -D hardhat @nomiclabs/hardhat-ethers @nomiclabs/hardhat-waffle  
ethereum-waffle ethers@5.7.0 dotenv chai @openzeppelin/contracts  
@nomicfoundation/hardhat-verify
```

* Note I am using Ethers version 5.7.0 because Ethers version 6 has been having a lot of issues with the Sepolia network (a testnet we will be deploying the smart contract to).

5. Once the installation is complete, we will run the hardhat module setup:

```
$ npx hardhat init
```

* Select the "Create an empty hardhat.config.js" option from the list.

This generates the hardhat config file and initializes the project as a hardhat project

6. Head over to the Hardhat website for the hardhat config setup:

Below is my local setup

```
/** @type import('hardhat/config').HardhatUserConfig */

require('@nomiclabs/hardhat-waffle');
require("dotenv").config();
require("@nomicfoundation/hardhat-verify");
const { task } = require('hardhat/config');
task('accounts', 'Prints the list of accounts', async (taskArgs, hre) => {
  const accounts = await hre.ethers.getSigners();
  for (const account of accounts) {
    console.log(account.address);
  }
});
module.exports = {
  networks: {
    hardhat: {
      gasPrice: 470000000000,
      chainId: 43112,
    },
    sepolia: {
      url: `${process.env.SEPOLIA_URL}`,
      accounts: [`${process.env.PRIVATE_KEY}`]
    },
    mainnet: {
      url: `${process.env.MAINNET_URL}`,
      accounts: [`${process.env.PRIVATE_KEY}`],
      chainId: 1
    }
  },
  solidity: {
    version: "0.8.27",
    settings: {
      optimizer: {
        enabled: true,
        runs: 200
      }
    }
  },
  etherscan: {
    apiKey: `${process.env.ETHERSCAN_API}`
  },
  sourcify: {
    enabled: true
  },
  paths: {
    sources: "./contracts",
    tests: "./test",
    cache: "./cache",
    artifacts: "./artifacts"
  },
  mocha: {
    timeout: 40000
  }
};
```

7. Lets get these URLs (Sepolia, for testing, and Mainnet, for live deployment).

Create an account or signin to the Alchemy Platform (we need Alchemy for the access to the Ethereum Nodes (mainnet or testnets).

- a) Once logged in, right on the `Overview` tab, click on `Create new app`
- b) Fill in the details of the app as preferred (Name, Description, Use case)
 - i. Name: Todo App
 - ii. Description (optional): dApp for Web3 Tutorial

- iii. Use case: Other project
 - iv. Other project type: Utility
 - c) Click on Next
8. ON the Next Page, you'll be asked to `Choose chains`
- a) Select Ethereum
 - b) Click Next
9. The Last page is the `Activate services` page.
- a) It has default 4 services selected. You can leave these default or add more (I am accepting the default 4 selections (TokenAPI, NFT API, Webhooks, Node API)
 - b) Click `Create app`
10. An API key will be generated for you with a sample code ussage.We will be adding this API KEY to our .env file (as we have imported the values into the **hardhat.config.js** file earlier).
11. Create a .env file at the root of the smart-contract project dir. Add the following variables in it:

```
SEPOLIA_URL=  
MAINNET_URL=  
API_KEY=  
  
PRIVATE_KEY=  
  
ETHERSCAN_API=  
  
CONTRACT_ADDRESS=
```

12. In the sample codes given, select the SDK as ethers.js, then you'll see a link in the `JsonRpcProvider` (the default network is the Mainnet innet), copy this link.

← Apps

Todo App

API Key 3 [redacted] Copy

Set up Metrics Networks Settings

Integrate your app

Docs

Chain: Ethereum Network: Mainnet Method: eth_getBlockByNumber - get the latest... Language: JS JavaScript SDK: ethers.js

Copy the code below into your code editor

```
Request  
  
import { JsonRpcProvider } from 'ethers';  
  
// Connect to the Ethereum network  
const provider = new JsonRpcProvider("https://eth-mainnet.g.alchemy.com/v2/3[redacted]");  
  
// Get block by number  
const blockNumber = "latest";  
const block = await provider.getBlock(blockNumber);
```

13. Paste the link in the .env file as the `MAINNET\_URL`

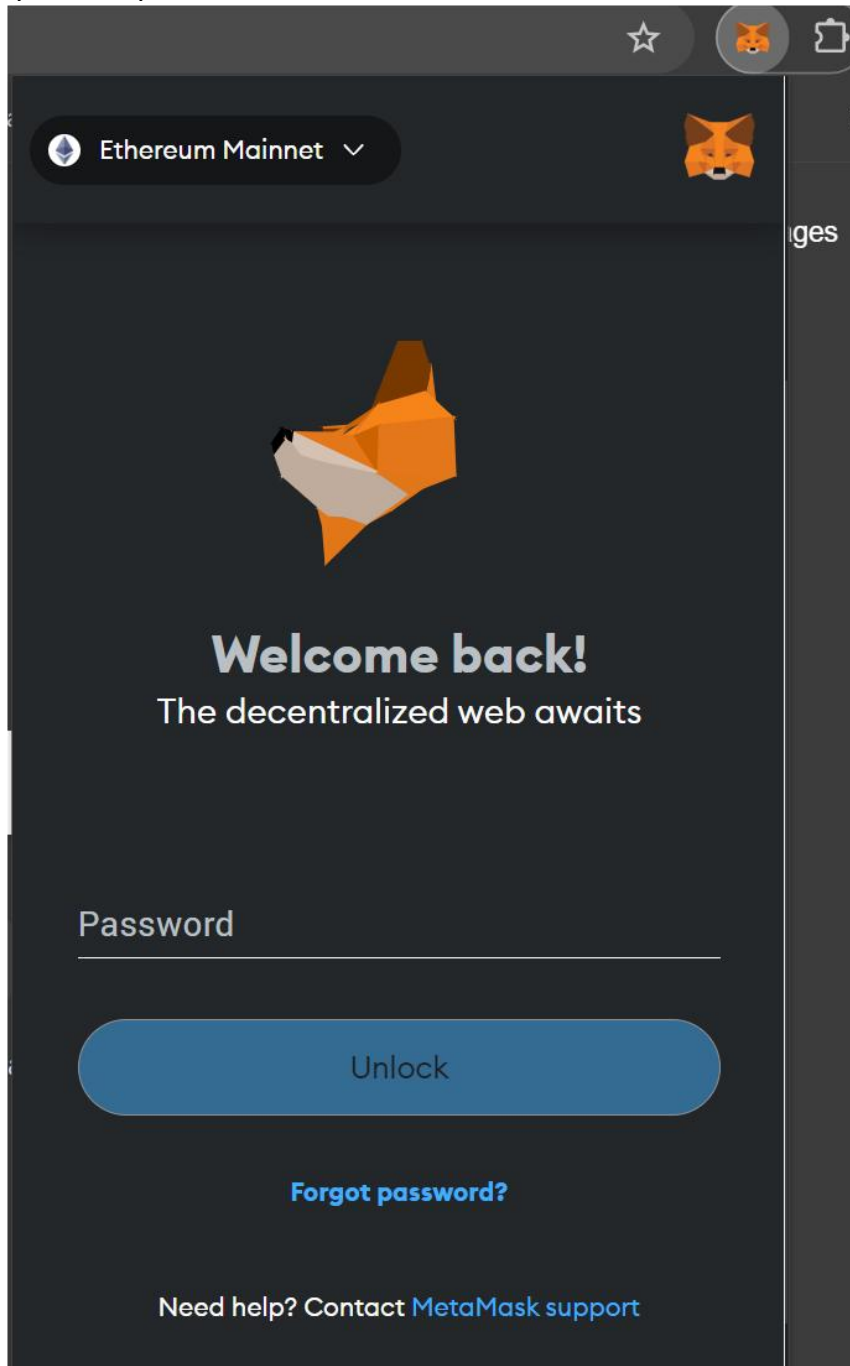
14. Click on the Network options (Under Integrate your app), and select Sepolia, Then copy the link inside the JsonRpcProvider (just like we did for the Mainnet).

15. Open your Ethereum wallet app (You are required to have one installed). I am using Metamask.

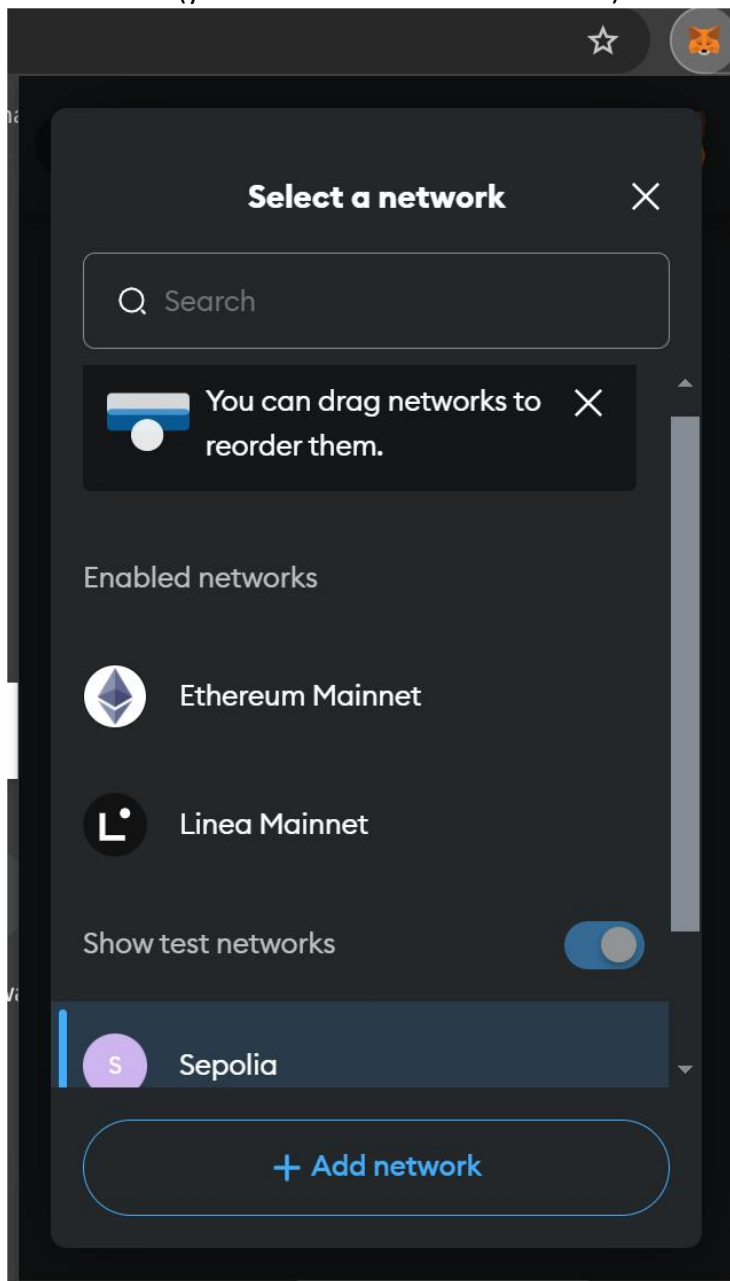
a) Once installed, create your wallet and you'll be given a public and private keys.

b) Copy the private key and add it to the .env PRIVATE_KEY variable

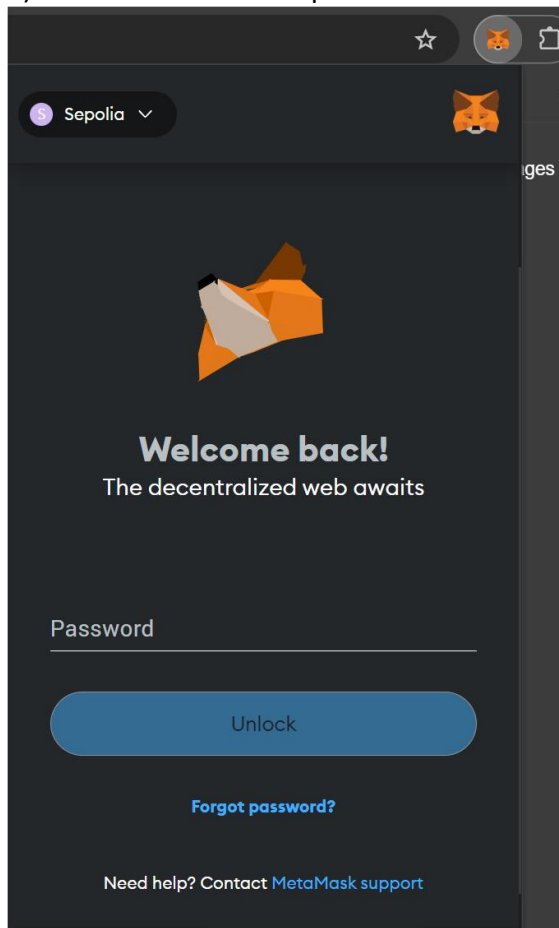
16. By default your metamask should be connected to the Mainnet.



- a) Click on the `Ethereum Mainnet` and you'll see a page to select your desired network (you can search for it if not found)



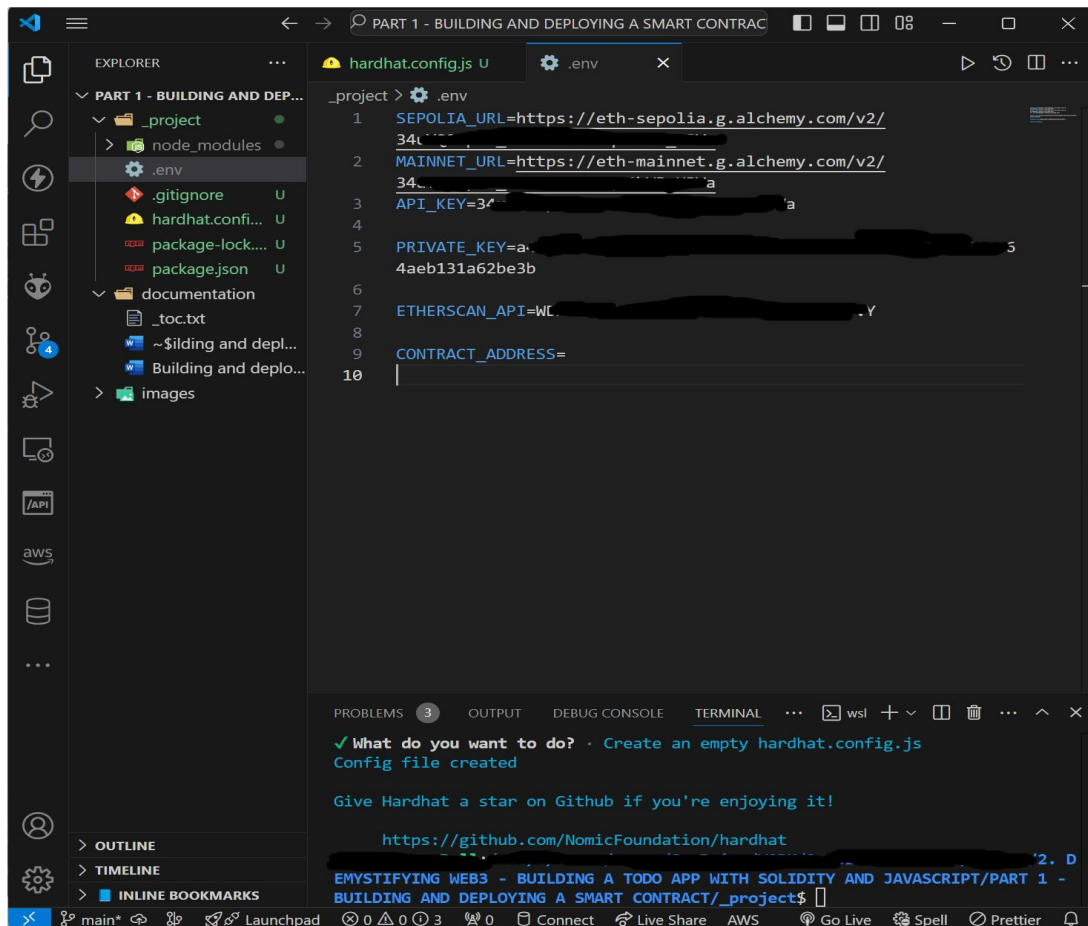
b) Select or click on `Sepolia` to choose the sepolia network



17. Head over to any Sepolia faucet and get your free Sepolia testnet tokens (you'll need this to make transactions on the Sepolia network)

18. Register on Etherscan, and get an API KEY

19. Our .env file should look like this



20. Create the subproject folders

- a) contracts
- b) scripts

21. Its finally time to create our Solidity contract.

Create a new file todo.sol inside the **contracts** directory. This is where we will create our smart contract codes for the TODO dApp.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.27;

contract TodoApp {
    // structure to define a Task
    // this holds a single task object
    struct Task {
        uint256 id;
        string taskDescription;
        bool isCompleted;
    }

    // Mapping of task id to the Task structure
    // This is used to create a 'list' of tasks that can be indexed
    // e.g 'tasks[taskId]' // where each taskId represents a Task
    mapping(uint256 => Task) public tasks;
    // Counter to auto generate task ids
    // this is like a sql database primary key that is used to hold a single row of record
    uint256 public taskCount;
}
```

```

// EVENTS
// event to emit when a task is created
event TaskCreated(uint256 id, string taskDescription, bool isCompleted);
// event to emit when a task is marked as completed
event TaskCompleted(uint256 id, bool isCompleted);
// event to emit when a task is deleted
event TaskDeleted(uint256 id);

// Constructor to initialize the first task id
constructor() {
    taskCount = 0;
}

// function to create a net task
function createTask(string memory _taskDescription) public {
    // on every new task increase the task count
    taskCount++;
    // add the new task to the list of tasks
    tasks[taskCount] = Task(taskCount, _taskDescription, false);
    // emit the event for new task creation
    emit TaskCreated(taskCount, _taskDescription, false);
}

// function to mark a task as completed
function completeTask(uint256 _taskId) public {
    require(_taskId > 0 && _taskId <= taskCount, "Invalid task ID"); // check if the task is available or has been created

    // create an instance of the task by selecting the task from the tasks list/array
    Task memory _task = tasks[_taskId];
    // change the task's isCompleted state to true
    _task.isCompleted = true;
    // replace the task in the task list with the updated task
    tasks[_taskId] = _task;
    // emit the task completed event
    emit TaskCompleted(_taskId, true);
}

// function to delete a task
function deleteTask(uint256 _taskId) public {
    require(_taskId > 0 && _taskId <= taskCount, "Invalid task ID"); // check that the given taskId is within the number of
tasks available

    delete tasks[_taskId]; // call the delete method (built-in) on the tasks list/array
    // emit event to broadcast that a task was deleted
    emit TaskDeleted(_taskId);
}

// function to get individual task details
function getTask(uint256 _taskId) public view returns (uint256, string memory, bool) {
    require(_taskId > 0 && _taskId <= taskCount, "Invalid task ID"); // check to confirm if the _taskId exists

    Task memory _task = tasks[_taskId]; // create a new instance of the task
    return (_task.id, _task.taskDescription, _task.isCompleted);
}

// function to get all tasks available
function getAllTasks() public view returns (Task[] memory) {
    require(taskCount > 0, "No tasks yet"); // make sure we have tasks created already

    // create a new list/array of tasks called `allTasks` from the Task structure we created in the beginning
    // the length of the Tasks array/list is set to the total count in the contract: `Task[](taskCount)`
    Task[] memory allTasks = new Task[](taskCount);

    for (uint256 i = 1; i <= taskCount; i++) {
        allTasks[i - 1] = tasks[i]; // reassigning the all tasks index so it begin at 0
    }
    return allTasks;
}
}

```

22. Lets create our JavaScript deploying script. This script will help us deploy our smart contract to the network

Inside the `scripts` folder, create the deploy.js file.

```
const hre = require("hardhat"); // import the hardhat module
require("dotenv").config(); // we can use this to import environmental variables

// function to deploy the smart contract to the network
const main = async () => {
  // this line of code gets the "TodoApp" smart contract.
  // the ethers.getContractFactory is used to prepare the contract for deployment by getting its bytecode and ABI
  const TodoApp = await hre.ethers.getContractFactory("TodoApp"); // check your smart contract codes again, you'll see we named the contract "TodoApp"

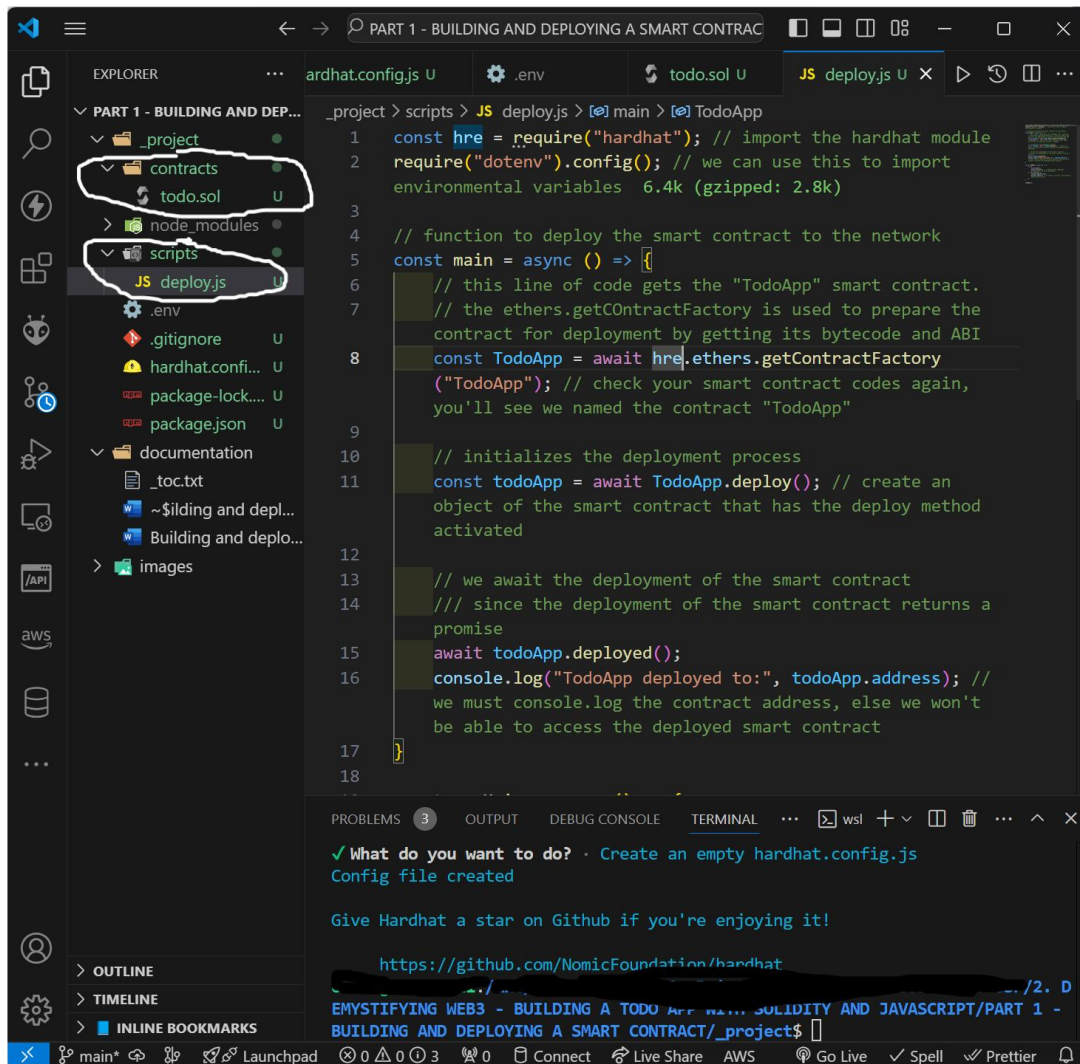
  // initializes the deployment process
  const todoApp = await TodoApp.deploy(); // create an object of the smart contract that has the deploy method activated

  // we await the deployment of the smart contract
  /// since the deployment of the smart contract returns a promise
  await todoApp.deployed();
  console.log("TodoApp deployed to:", todoApp.address); // we must console.log the contract address, else we won't be able to access the deployed smart contract
}

const runMain = async () => {
  try {
    await main();
    process.exit(0); // if the process completes successfully, we exit gracefully
  } catch (error) {
    console.log({error});
    process.exit(1); // if there's an error, the function exits with an error.
  }
}

runMain();
```

23. Your new folder structure should look like this



24. Compile the newly created smart contract. Run the command below

```
$ npx hardhat compile
```

This command checks through the Solidity smart contract codes for errors.

If the code verification process is success, the hardhat module will then compile the Solidity code.

This process will take a little while, so relax and wait for the completion message on the terminal

```
`Compiled 1 Solidity file successfully (evm target: paris).`
```

You will get this kind of message in the terminal if there's a successful compilation.

Else, you'll get an error message like the one below (stating where the error is).

```
BUILDING AND DEPLOYING A SMART CONTRACT/_project$ npx hardhat compile
ParserError: Expected ';' but got '}'
--> contracts/todo.sol:91:9:
    |
91 |         }
    |         ^

Error HH600: Compilation failed

For more info go to https://hardhat.org/HH600 or run Hardhat with --show-stack-traces
```

25. Now our TodoApp smart contract is ready to be deployed to the Network (Sepolia test network).

4. DEPLOY SOLIDITY SMART CONTRACT

4.1 Deploy our TodoApp smart contract to the Sepolia Testnet (Using Hardhat on the CLI)

I. Run the command below to deploy our TodoApp smart contract to the Sepolia Testnet:

\$ npx hardhat run scripts/deploy.js --network sepolia

Once the contract has been successfully deployed, you'll get a message printed on the terminal about the contract address of the deployed contract

```
NG A TODO APP WITH SOLIDITY AND JAVASCRIPT/PART 1 - BUILDING AND DEPLOYING A SM
npx hardhat run scripts/deploy.js --network sepolia
TodoApp deployed to: 0x063C80E7D35004feA780fA52cEDa4986d6FC95d3
```

Select the entire message (TodoApp deployed to:

0x063C80E7D35004feA780fA52cEDa4986d6FC95d3) and copy it.

* In my case, when I copy only the contract address, it gives a wrong hex. I am using Windows OS (11), so in case you have the same error, just copy the entire deployed message. Nothing else worked.

II. Paste the contract address into your .env file for the variable CONTRACT_ADDRESS

III. The deployment process creates 2 additional folders in the project director

- a. artifacts
- b. cache

These folders hold the deployment information for the smart contract (like the contract abi that we need to interact with the smart contract).

4.2 Verifying our Smart Contract on the Etherscan using the CLI

Run the command below to verify smart contract on Etherscan:

```
$ npx hardhat verify --network networkname contract_address deployment_message
npx hardhat verify --network sepolia
0x063C80E7D35004feA780fA52cEDa4986d6FC95d3
```

In our case, we don't have a deployment message.

You'll get a verification response on the terminal that shows the state of the verification process (whether a success or failure).

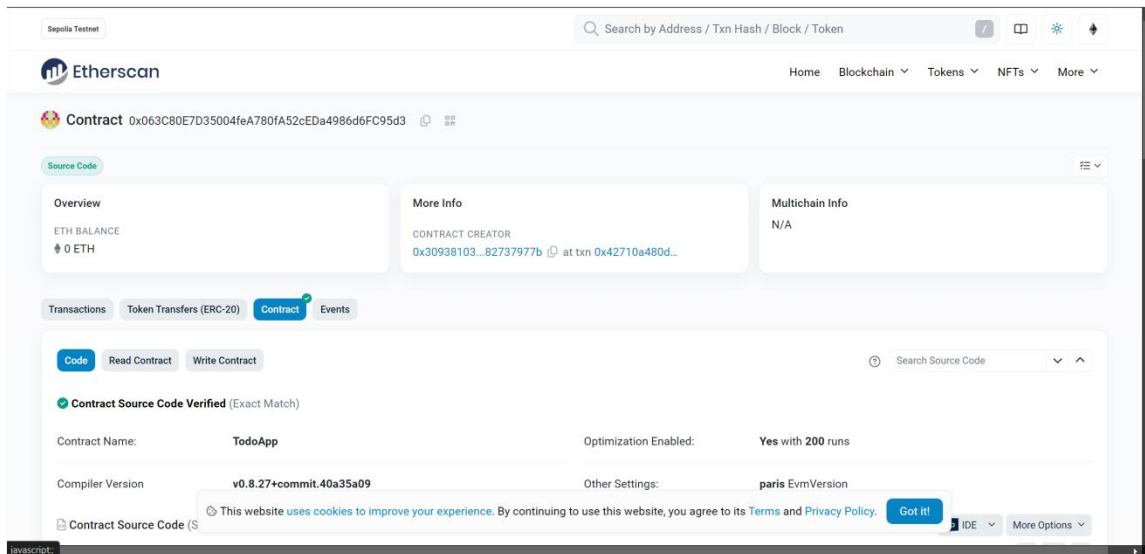
```
NG A TODO APP WITH SOLIDITY AND JAVASCRIPT/PART 1 - BUILDING AND DEPLOYING A SMART CONTRACT/_projects$
npx hardhat verify --network sepolia 0x063C80E7D35004feA780fA52cEDa4986d6FC95d3
Successfully submitted source code for contract
contracts/todo.sol:TodoApp at 0x063C80E7D35004feA780fA52cEDa4986d6FC95d3
for verification on the block explorer. Waiting for verification result...

Successfully verified contract TodoApp on the block explorer.
https://sepolia.etherscan.io/address/0x063C80E7D35004feA780fA52cEDa4986d6FC95d3#code

Successfully verified contract TodoApp on Sourcify.
https://repo.sourcify.dev/contracts/full_match/11155111/0x063C80E7D35004feA780fA52cEDa4986d6FC95d3/
```

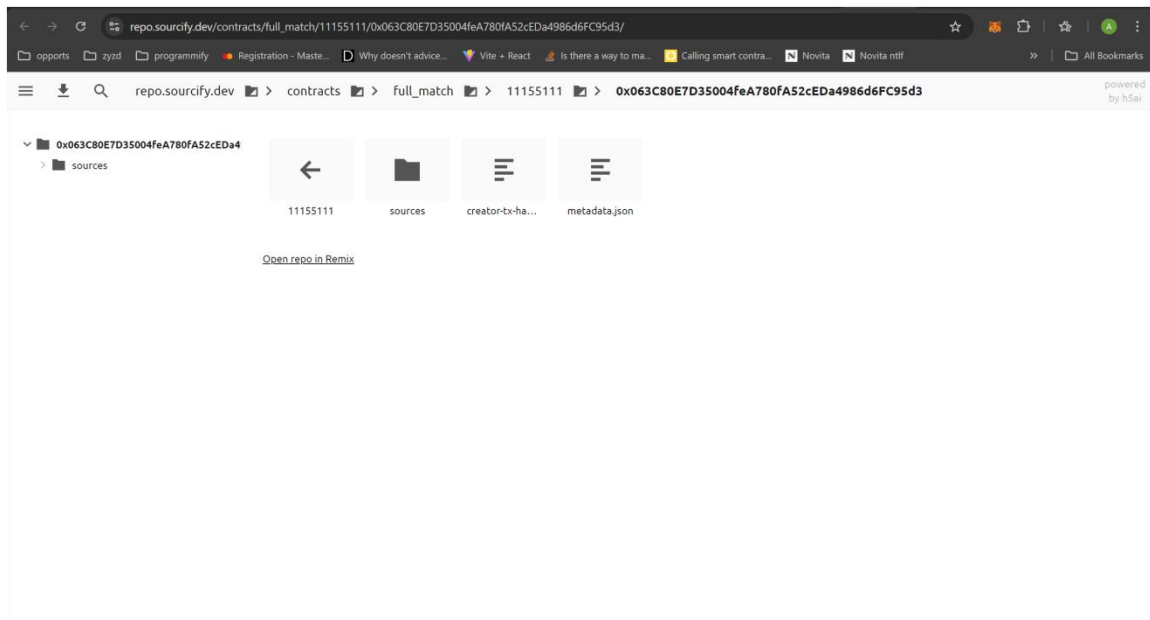
4.3 Verifying our Smart Contract on the Etherscan block explorer using the Browser.

I. Click on the Etherscan block explorer for the Sepolia network, displayed in the terminal of the verification step,
(<https://sepolia.etherscan.io/address/0x063C80E7D35004feA780fA52cEDa4986d6FC95d3#code>).

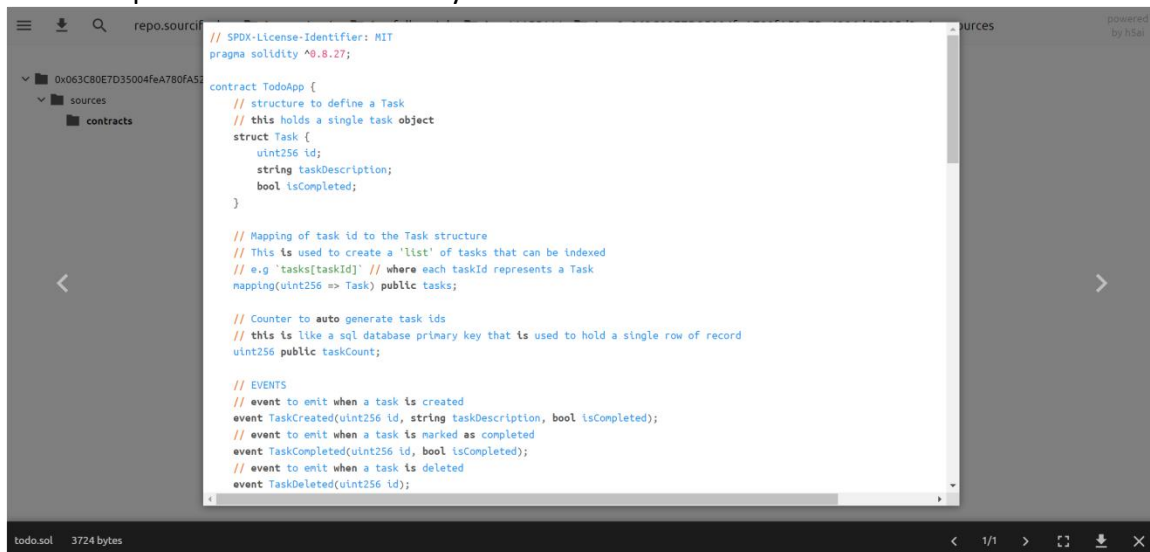


If you scroll down, you'll see the source code for the TodoApp contract.

II. Click on the second link, the sourcify link
(https://repo.sourcify.dev/contracts/full_match/11155111/0x063C80E7D35004feA780fA52cEDa4986d6FC95d3/), and you'll be taken to the sourcify link for the project



You can open the sources directory to check the source codes for the contract too.



These methods can be used to verify source codes of smart contracts (for vulnerability checks and audits) by ensuring that the deployed bytecodes matches the provided source codes.

5. CONCLUSION

5.1 Interacting with our TodoApp Smart contract on the Backend

Congratulations. So far you have created, deployed, and verified your TodoApp smart contract.

But we need to test it (by interacting with it).

Lets do that.

STEPS:

1. In our scripts folder, create a new file `interact.js`

Create the JavaScript codes to interact with the smart contract deployed

```
require("dotenv").config();
const hre = require("hardhat");
const { ethers } = require("ethers"); // ethers v6 have given me a lot of issues for Sepolia network, so I use ethers v5.7.0
const contract = require("../artifacts/contracts/todo.sol/TodoApp.json");

const SEPOLIA_URL = process.env.SEPOLIA_URL;
const PRIVATE_KEY = process.env.PRIVATE_KEY;
const CONTRACT_ADDRESS = process.env.CONTRACT_ADDRESS;

// Provider (a node (provider) that allows interaction (read and write access) to the blockchain)
const provider = new ethers.providers.JsonRpcProvider(SEPOLIA_URL);

// Signer (this is the account that will be interacting with the blockchain (or has the right to sign transactions))
const signer = new hre.ethers.Wallet(PRIVATE_KEY, provider);

// TodoApp Contract (deployed) (this is the contract deployed to the blockchain, that we want to interact with. it is an Ethers.js object that is on-chain.)
const todoAppContract = new hre.ethers.Contract(CONTRACT_ADDRESS, contract.abi, signer);

// get all todos
const getAllTodos = async () => {
  try {
    const todos = await todoAppContract.getAllTasks();
    console.log("Todos in the smart contract:", todos);
  } catch (error) {
    console.log("Error while fetching all todos:", error);
  }
}

// create new todo
const createTodo = async (todoDescription) => {
  try {
    await todoAppContract.createTask(todoDescription);
    console.log("New todo task created successfully.");
  } catch (error) {
    console.log("Error while adding new todo:", error);
  }
}

// delete todo
const deleteTodo = async (todoId) => {
  try {
    await todoAppContract.deleteTodo(todoId);
  } catch (error) {
    console.log("Error while deleting todo:", error);
  }
}
```

```
// lets run the functions
getAllTodos();
// createTodo("Hello Web3");
// deleteTodo(1);
```

We are calling the different functions depending on what we want to achieve or what we want the smart contract to do.

2. To run the `interact.js` file, use `node` to run it.

```
$ node scripts/interact.js
```

[illegible]

As you can see, we get an error that 'No tasks yet', this was the response we set in our contract to return when there's no tasks added yet.

3. So lets run a function to create a new task:

```
// lets run the functions
// getAllTodos();
createTodo("Hello Web3");
// deleteTodo(1);
```

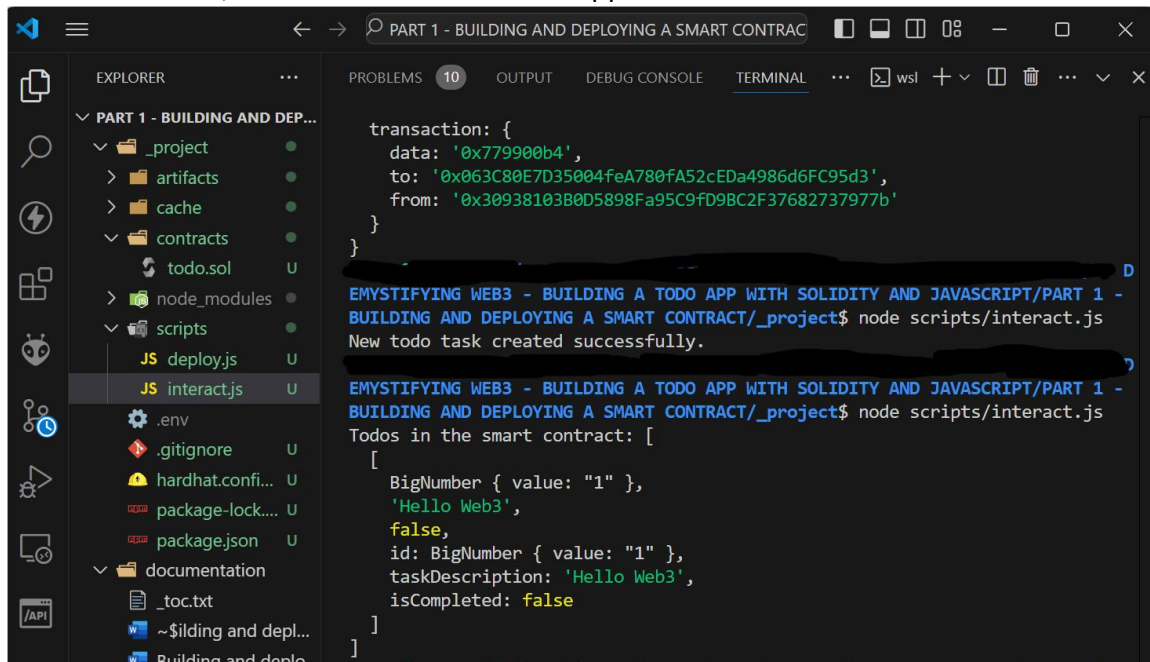
Great, we got a success message for the createTodo function call:

```
BUILDING AND DEPLOYING A SMART CONTRACT/_project$ node scripts/interact.js
New todo task created successfully.
```

4. We can re-run the getAllTodos function now to see all the created todos

```
// lets run the functions
getAllTodos();
// createTodo("Hello Web3");
// deleteTodo(1);
```

There we have it, our final test for the TodoApp Smart contract backend flow result:

The screenshot shows the Visual Studio Code interface with a project titled 'PART 1 - BUILDING AND DEPLOYING A SMART CONTRACT'. The Explorer panel on the left shows the project structure, including folders like '_project', 'artifacts', 'cache', 'contracts', and 'scripts', along with files like 'todo.sol', 'node_modules', 'scripts', 'JS deploy.js', 'JS interact.js', '.env', '.gitignore', 'hardhat.config.js', 'package-lock.json', 'package.json', 'documentation', '_toc.txt', and a file named '~\$ilding and depl...'. The Terminal panel on the right shows the output of a command. It displays a transaction object with data, to, and from addresses. Below this, it shows the command 'node scripts/interact.js' being executed, which results in a successful creation of a new todo task. The terminal output also shows the current state of the smart contract, listing a single todo item with a value of '1', description 'Hello Web3', and a status of 'false' for 'isCompleted'.

5.1 Summary and Conclusion

Web3 represents a new and modern phase of the WWW, and it leverages the Blockchain distributed Ledger Technology. Web3 prioritizes ownership, privacy, and decentralization.

It is expected that Web3 dApps will disrupt the traditional models of online transactions and interactions and by learning to build and interact with Web3 technologies, you are stepping into the future of the internet.

5.2 What's Next (Part 2: Connecting to our TODO Smart Contract from the Client Side using React, JavaScript, and Web3.js)

Great! We have successfully completed our task of creating, deploying, verifying, and interacting with a smart contract (a TodoApp smart contract).

But, our users can not be expected to have all these setups for interactions and be expected to run codes everytime they want to use the smart contract.
Hence, we will need to build a User Interface - Web3, that will be connected to the Smart contract, and our users can then interact with the Todo Smart contract using our UI (Web3 client).
See you then!

Best regards and wishes as we conquer the Web3 Space!
Abdulahakeem Muhammed
(Blockchain/Cryptocurrency/Smart-Contract Developer, Software Engineer
(full stack), Data Engineer, AI/ML Engineer)